
KernelMind: The Hybrid Meta-Scheduler

Architecting an Intelligent CPU Dispatcher

Introduction: The Blueprint

An Operating System's CPU Scheduler faces a relentless challenge: managing unpredictable user workloads while minimizing system latency. Traditional schedulers rely on hardcoded heuristics like Round Robin (RR) or Shortest Job First (SJF). These static heuristics force a perpetual compromise between blazing-fast throughput and equitable fairness.

In this module, you will build a **Meta-Scheduler**. Rather than allowing an agent to manipulate raw CPU cycles directly (a deep-learning challenge reserved for our final KernelMind module), your agent will act as an intelligent orchestrator. You will provide it with a discrete set of traditional scheduling algorithms as its "action space." Your objective is to train a Reinforcement Learning agent to dynamically switch heuristics on the fly, evaluating queue conditions to synthesize a hybrid policy that balances throughput and fairness.

Part 1: Architecture & Baseline Simulators

Before introducing Artificial Intelligence, you must build the traditional OS simulation architecture. A clean, modular, object-oriented approach is highly recommended.

The Convoy Effect

Early operating systems utilized FCFS (First-Come, First-Served) logic. This led to a devastating, cascading failure known as the *Convoy Effect*. Imagine a single, massive, CPU-bound process (like compiling a huge binary file) monopolizing the CPU. Meanwhile, I/O-bound processes (like handling user keystrokes, mouse clicks, or network packets) that only require fractions of a millisecond are trapped waiting in line. The CPU utilization might technically be high, but the interactive response time dies, making the system feel completely frozen to the user.

Your Task:

1. **Process Representation:** Create a system to track a process's arrival time, burst time, remaining time, start time, and finish time.
2. **Workload Generation:** Write a generator that spawns an array of random processes (e.g., 10 processes with randomized burst times and staggered arrivals) to serve as a testing "episode."
3. **Baseline Simulators:** Implement independent simulation loops for the following baselines:
 - **Random Agent:** Selects a scheduling heuristic completely at random. This serves as the absolute minimum performance floor.
 - **FCFS (First-Come, First-Served):** Executes processes strictly in the order they arrive.

- **SJF (Shortest Job First):** Non-preemptive. Selects the process with the smallest total burst time.
- **Round Robin (RR):** Preemptive. Executes processes for a fixed time slice (quantum) before cycling to the next process.

Design Question 1: The Preemption Paradigm

Your SJF baseline is non-preemptive. Research its preemptive counterpart: **Shortest Remaining Time First (SRTF)**. Conceptually, why is preemption mathematically necessary to achieve the absolute lowest possible Mean Wait Time? What flaw occurs in a non-preemptive SJF system if a massive database backup begins running exactly 1 tick before a dozen tiny web requests arrive?

Part 2: The Evaluation Metrics

To benchmark your agent against the baselines, you need a robust evaluation suite. An OS cannot be judged on speed alone; it must also be fair.

Your Task: Create a utility to calculate the following metrics from a list of completed processes:

- **Mean Wait Time:** The primary indicator of system throughput.
- **P90 Wait Time:** The wait time experienced by the 90th percentile of processes (exposing worst-case latency).
- **Jain's Fairness Index:** A mathematical formula ranging from $\frac{1}{n}$ (worst) to 1.0 (perfect fairness) that measures how equally the wait times are distributed among jobs.

Part 3: Formulating the Environment

Now, translate your CPU simulator into a Markov Decision Process (MDP) so the agent can interact with it.

Starvation and Aging

Algorithms like SJF guarantee theoretically optimal mean wait times, but they inherently discriminate against long-running processes. If a steady stream of short tasks keeps arriving, a large task is permanently "starved" at the back of the queue. To counter this, real-world OS architects invented *Aging*—a mechanism that artificially inflates a waiting process's priority based on how long it has been in the queue. Your RL agent will need a mathematical equivalent to Aging in its reward function to learn this fairness behavior naturally.

The Context Switch Penalty

If FCFS causes the Convoy Effect, why not preempt processes every single millisecond? At the hardware level, pausing a process requires the OS kernel to save CPU registers, flush cache lines, and load a new Process Control Block (PCB). If the time slice (quantum) is too small, the system spends more computing power switching contexts than doing actual work—a state known as *thrashing*.

Your Task: Design an Environment class with standard `reset()` and `step(action)` methods.

- **State Space:** Discretize the state of the queue (e.g., bucketing the current Queue Length and the Average Burst Time) into integer indices so they can map to a tabular Q-table.
- **Action Space:** Map discrete integer actions to the execution heuristics you wrote in Part 1.
- **Reward Function:** Formulate a reward signal that penalizes the agent for poor performance. You must balance the penalty for total queue delay against the penalty for starving individual long jobs.

Design Question 2: Reward Shaping

Explain the mathematical logic behind your chosen reward function. If you design a simple linear penalty, an agent might realize that indefinitely starving one massive job saves more aggregate waiting time for the smaller jobs than the single penalty costs it. How did you mathematically structure your reward to prevent the agent from justifying starvation?

Part 4: The Meta-Agent

Build a Tabular Q-Learning agent to navigate the environment.

Approximating the MLFQ

In the real world, production operating systems (like Linux, macOS, and Windows) do not strictly use SJF or RR. They rely on Multi-Level Feedback Queues (MLFQ). An MLFQ uses an array of distinct queues with different priority levels and quantum sizes, relying on complex, hand-tuned logic to demote CPU hogs and promote interactive tasks. By giving your RL agent access to discrete scheduling algorithms, you are challenging it to natively approximate a dynamic MLFQ without having to hardcode the transition rules yourself.

Your Task:

- Implement a Q-table initialization based on the dimensions of your environment's state space and action space.
- Write the Bellman update logic to adjust action values based on the Temporal Difference error.
- Implement an ϵ -greedy action selection policy. *Note: You must manage the decay of ϵ carefully. OS scheduling requires absolute precision; a single random exploration action late in*

training can skew an entire episode's metrics. Ensure your decay logic eventually enforces a "hard cutoff" to pure exploitation.

Part 5: Mission Control & Visualizing Convergence

Establish a training and plotting pipeline to prove your Meta-Scheduler outperforms the static algorithms.

Your Task:

1. Run standalone evaluation episodes using strictly your baseline algorithms from Part 1 (including the Random Agent). Record their metrics.
2. Initialize your environment and agent, and execute a training loop for thousands of episodes (e.g., 20,000 to 50,000).
3. **Convergence Plots:** Use `matplotlib` to plot the agent's moving average wait time over the training episodes. Overlay this plot with flat, horizontal lines representing the baseline scores (FCFS, RR, SJF, Random) so you can visually prove the exact episode where your agent mathematically surpasses the industry standards.
4. Run evaluation metrics against the fully trained agent and output a comparative data table.

Design Question 3: Output Analysis

Analyze your final comparative data table and convergence plot. Did your agent successfully form a hybrid policy? How did its metrics compare to pure SJF and pure RR? Did it favor specific actions more heavily than others, and why do you think it made those choices based on your specific reward function?

Deliverables

Submit your codebase archive containing the following components (or encapsulated in a single notebook):

1. The process/workload generator logic and baseline algorithms.
2. The core MDP Environment and Tabular Q-Learning Agent architectures.
3. The evaluation and metrics suite.
4. The training execution script demonstrating your training loop.
5. A generated convergence plot demonstrating the RL agent's learning trajectory vs. baselines.
6. A comprehensive `report.pdf` explaining your code implementation, your answers to the 3 Design Questions, and a breakdown of how well your agent actually performed during training.

Optional Objectives

The Workload Stress Test

A robust OS must survive unpredictable scenarios. Currently, your workload generator likely distributes burst times and arrivals somewhat uniformly.

The Challenge: Create a secondary workload mode called "I/O Storm." In this mode, generate 8 incredibly short jobs (bursts of 1-2 ticks) that all arrive simultaneously alongside 2 massive CPU-heavy jobs (bursts of 50+ ticks).

Test your trained agent on this skewed distribution. Does the Q-table policy learned on a uniform workload successfully generalize to the I/O Storm, or does its Fairness index collapse?

The Context Switch Tax

In our current simulator, switching from one process to another happens instantly. In reality, the CPU must save registers and load new memory—a delay known as context switching overhead.

The Challenge: Modify your environment so that every time a preemptive action (like Round Robin) forces a context switch before a job is finished, a **1-tick penalty** is added to the system's total time. Retrain your agent. How does adding a strict mathematical cost to preemption change the agent's action distribution? Does it abandon Round Robin entirely, or just use it more carefully?

Dynamic Time Quanta

Real Multi-Level Feedback Queues don't just use one fixed time slice; they use small slices for high-priority interactive tasks and large slices for background tasks.

The Challenge: Expand your Action Space. Instead of a single Round Robin action, give the agent two: **RR-Short** (quantum = 2 ticks) and **RR-Long** (quantum = 8 ticks). Train this expanded agent. Check your logs to see if it learns to use RR-Short when average burst times are low, and RR-Long when average burst times are high.